Vulnerability Assessment Report

Prepared by: Noleen Naidoo

Future Interns - Cybersecurity Internship Candidate

Target Application: bWAPP (Buggy Web Application)

Target Application URL: http://bwapp.hakhub.net

Date: March 2026

1. Executive summary:

This report presents the results of a vulnerability assessment conducted on the intentionally vulnerable web application bWAPP hosted at http://bwapp.hakhub.net. The objective of the assessment was to identify potential security weaknesses using passive and non-intrusive security testing techniques.

The assessment included network reconnaissance, web application traffic analysis, and inspection of HTTP security configurations. During the assessment, several security issues were identified including exposed network services, server information disclosure, missing HTTP security headers, and insecure session cookie configurations.

Although automated security tools detected alerts of varying severity levels, this report focuses specifically on the four Medium-risk vulnerabilities that were verified during the assessment. These findings represent the most relevant security weaknesses that could potentially expose the application to common web-based attacks.

While the target application is intentionally vulnerable for educational purposes, the identified findings demonstrate security weaknesses that frequently occur in real-world web applications. Addressing these issues would significantly improve the overall security posture of the system.

2. Scope of assessment:

Item:	**Description:**
Target Application	bWAPP
Target URL	http://bwapp.hakhub.net
Assessment Type	Passive Web Vulnerability Assessment
Testing Method	Non-intrusive testing
Assessment Date	March 2026

3. Methodology:

The vulnerability assessment followed a structured methodology commonly used in security testing engagements. The process began with reconnaissance to identify open ports and exposed services using the Nmap network scanning tool.

Following reconnaissance, web traffic was analyzed using OWASP ZAP to perform a passive vulnerability scan. This allowed for the identification of potential security misconfigurations in HTTP responses without actively attacking or exploiting the application.

Additional analysis was conducted using browser developer tools to manually inspect HTTP response headers, cookies, and client-side security configurations. SecurityHeaders.com was also used to evaluate the presence of recommended HTTP security headers.

Only passive and read-only testing techniques were used during the assessment to ensure that the target system was not disrupted or negatively impacted. The results obtained from these tools were reviewed and verified, and the final report focuses specifically on the most relevant Medium-risk findings identified during the assessment.

4. Tools used:

Tool:	**Purpose:**
Nmap	Network scanning and service discovery
OWASP ZAP	Passive vulnerability scanning
Browser Developer Tools (Firefox/ Chrome Network Inspector)	Inspect HTTP headers and cookies
SecurityHeaders.com	Analyze HTTP security headers

5. Identified vulnerabilities:

1. Open Ports and Services	
Description:	A network scan of the target host revealed several open ports including HTTP (80), HTTPS (443), and port 5000. Open ports expose services that may be accessible to external users and attackers. The presence of unnecessary open services can increase the system's attack surface and provide additional entry points for exploitation.
Risk Level:	Medium
Recommendations:	Limit exposure of unnecessary services using firewall rules and restrict access to non-essential ports.
Evidence:	Screenshot of Nmap scan results. (Appendix 8.1)

2. Server Information Disclosure	
Description:	The HTTP response headers reveal details about the underlying web server and software environment. Specifically, the server identifies itself as nginx and discloses the PHP version through the X-Powered-By header. Exposure of such information may assist attackers in identifying potential vulnerabilities associated with specific software versions.
Risk Level:	Medium
Recommendations:	Disable server version disclosure and remove the X-Powered-By header.
Evidence:	Screenshot of Response Headers. (Appendix 8.2).

3. Missing Security Headers	
Description:	The application does not implement several recommended HTTP security headers. Missing headers such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options reduce protection against common web attacks including cross-site scripting (XSS) and clickjacking.
Risk Level:	Medium
Recommendations:	Implement recommended security headers at the web server or application level.
Evidence:	Screenshot from DevTools, OWASP ZAP, Response Headers. (Appendix 8.2 & 8.3).

4. Insecure Session Cookie Configuration	
Description:	The application uses a session cookie identified as PHPSESSID. The cookie does not include important security attributes such as Secure, HttpOnly, or SameSite. Without these protections, the session cookie may be exposed to client-side attacks or transmitted over insecure channels.
Risk Level:	Medium
Recommendations:	Configure cookies with the Secure, HttpOnly, and SameSite attributes to enhance session security.
Evidence:	Screenshot showing Cookie in request headers. (Appendix 8.4).

6. Risk classification:

Risk Level:	Number of Findings:
High	0
Medium	4
Low	0

7.Conclusion:

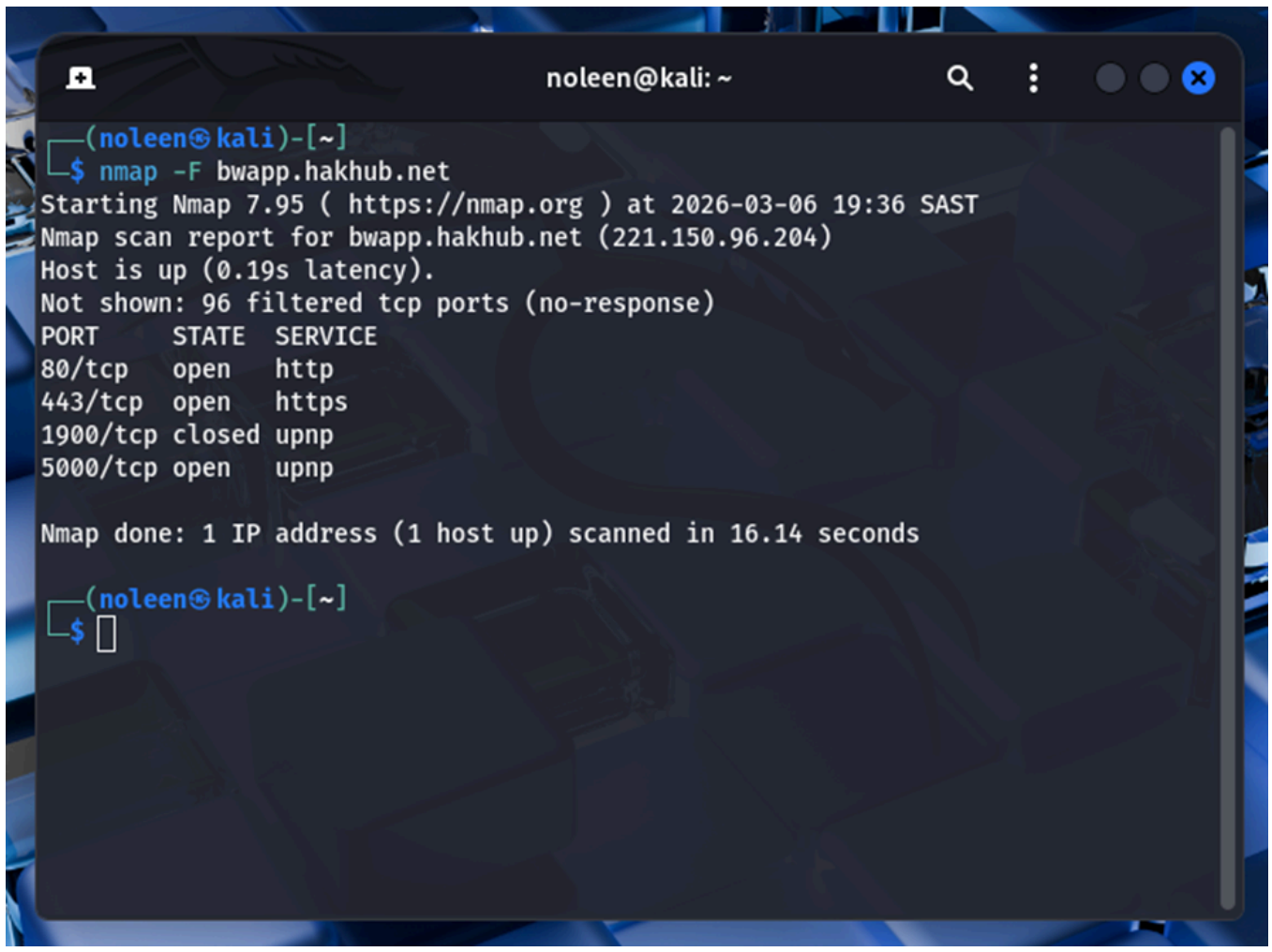
The vulnerability assessment identified four Medium-risk security weaknesses related to server configuration, information disclosure, and missing security controls within the tested web application. These vulnerabilities include exposed services, server information disclosure, missing HTTP security headers, and insecure session cookie configurations.

Although the application assessed is intentionally vulnerable for training purposes, the findings highlight common security weaknesses that are frequently encountered in real-world web environments. If left unaddressed, such vulnerabilities may increase the risk of information disclosure, session hijacking, or other web-based attacks.

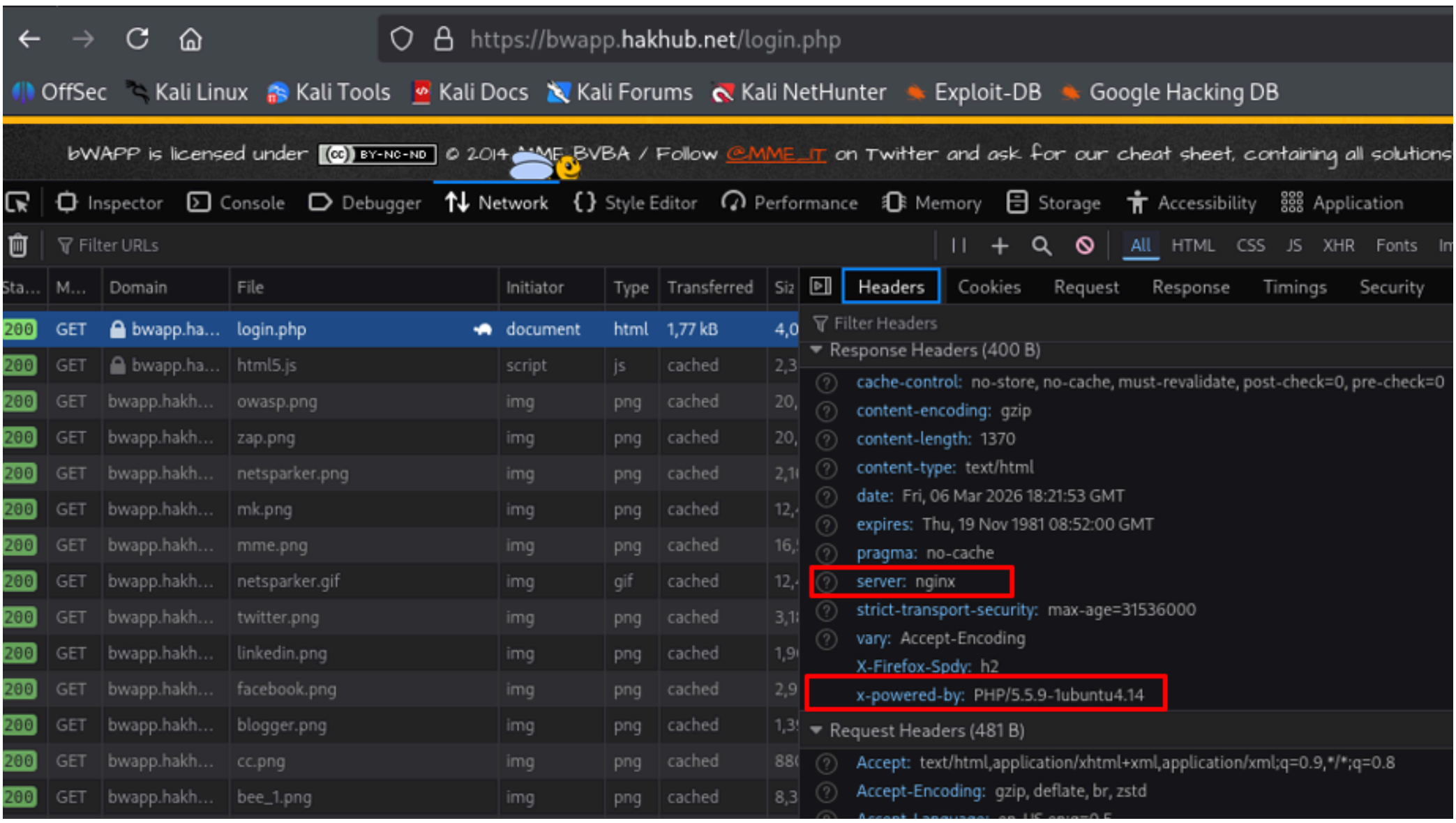
By implementing the recommended security improvements, such as restricting unnecessary services, removing sensitive server information from response headers, applying appropriate HTTP security headers, and strengthening session cookie configurations, the overall security posture of the application can be significantly improved.

8. Appendix (Evidence):

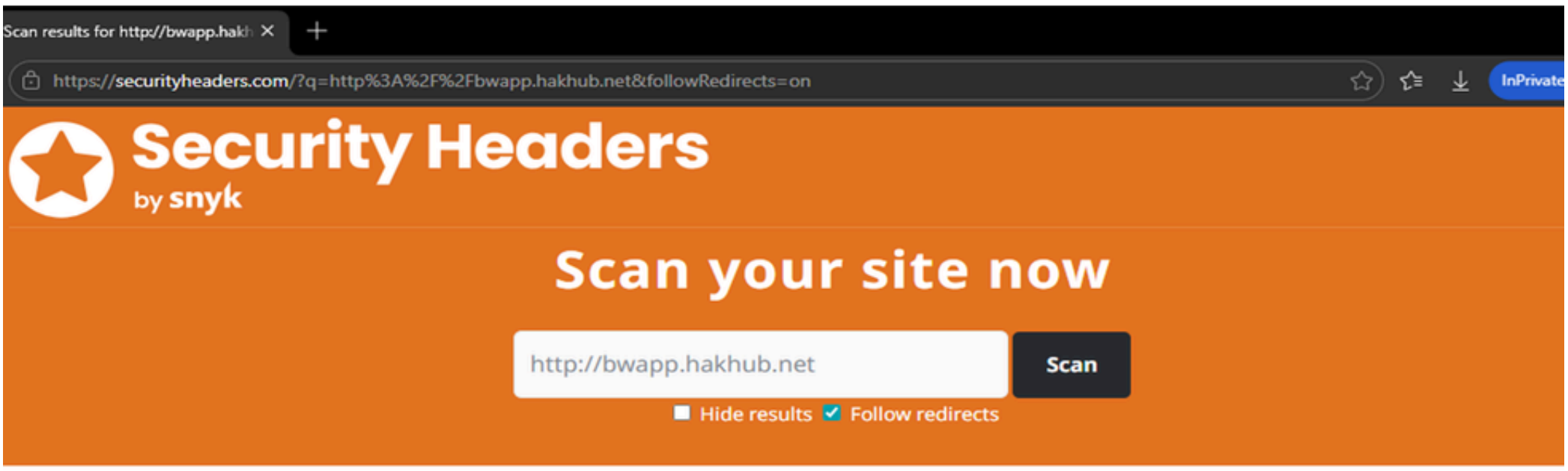
8.1: Open Ports and Services – Nmap Scan:

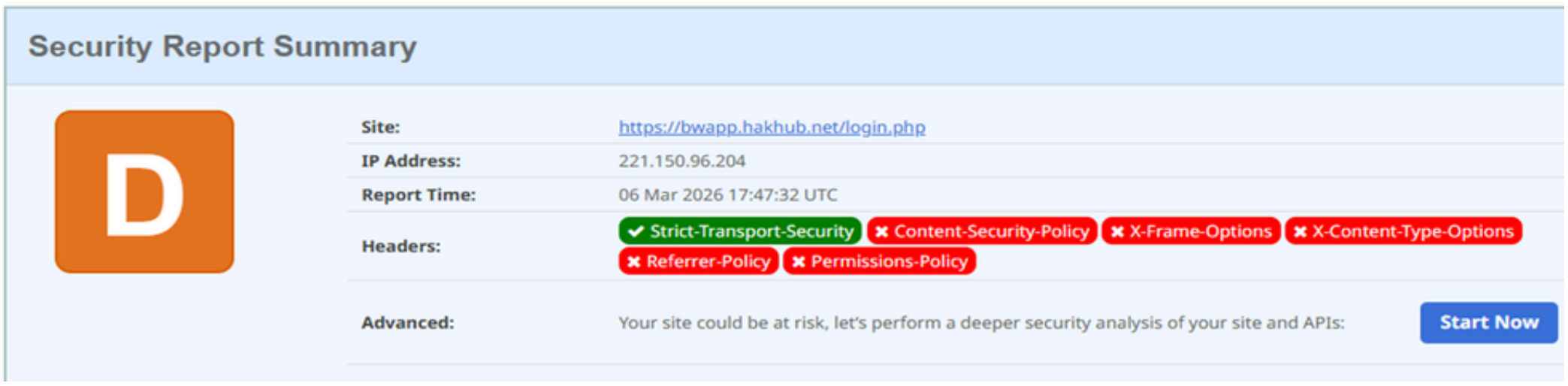


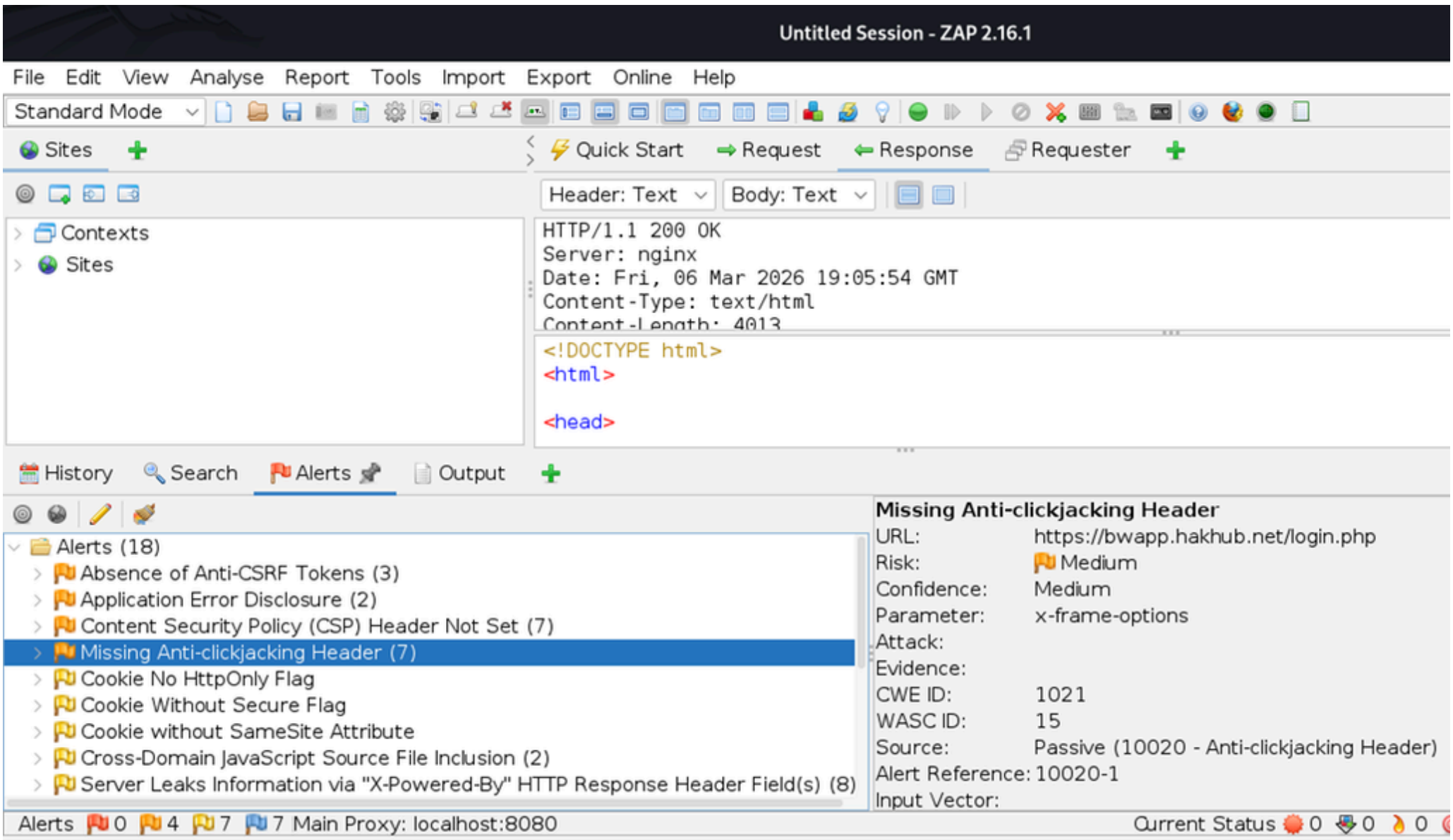
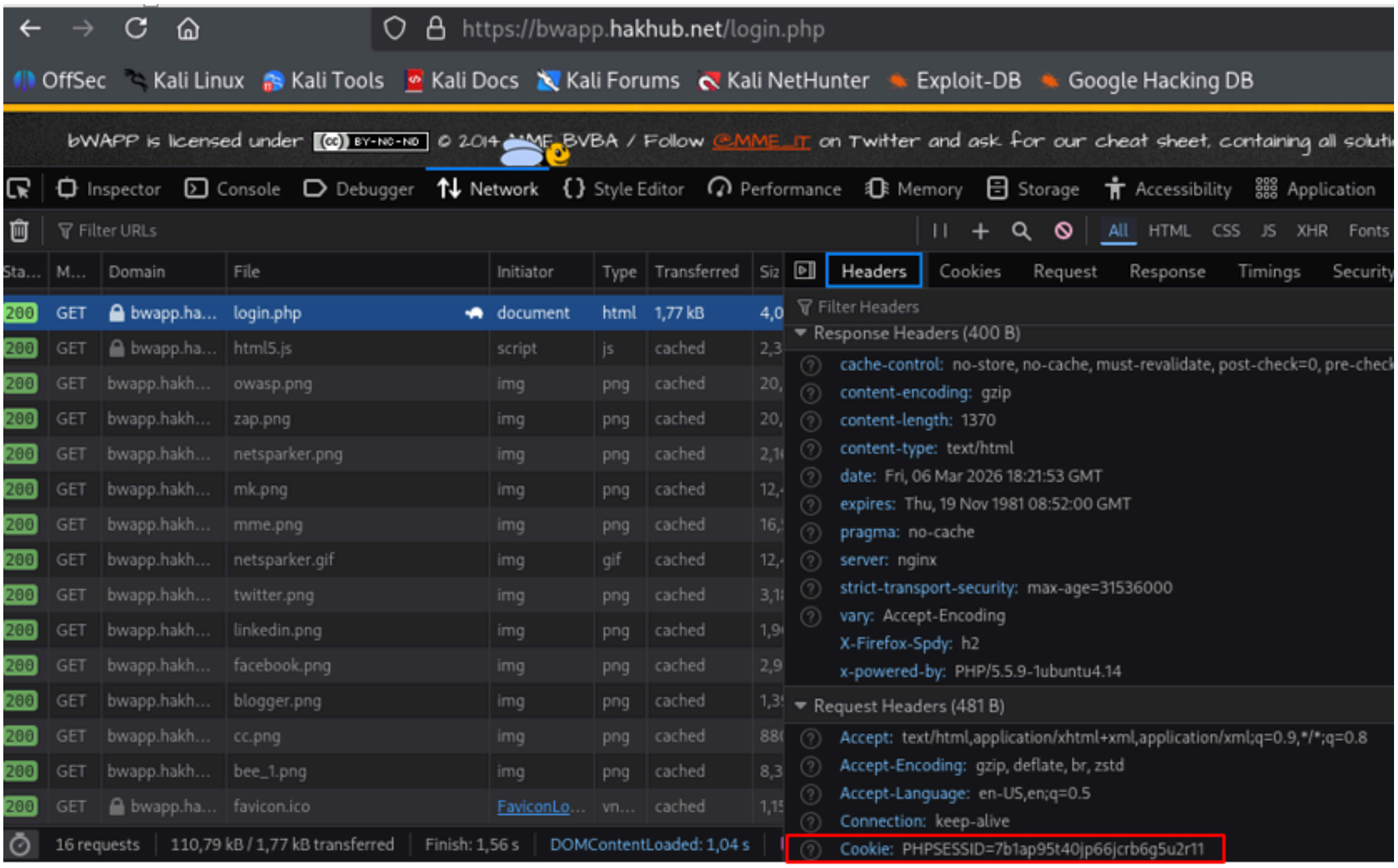
8.2. Server Information Disclosure:



8.3. Missing Security Headers:







8.4. Insecure Session Cookie Configuration